

AWS-Native MCP/RAG Platform: Porting AI-Assisted Development Infrastructure to Managed Cloud Services

From Docker Compose to Neptune, OpenSearch, and ECS Fargate
with Spec-Driven Development via Kiro

Terrence McGuinness
Modeling Development Center
National Weather Service, NOAA
Terry.McGuinness@noaa.gov

April 16, 2026

Abstract

We describe the design and implementation of a complete port of the MDC MCP/RAG Server—a 51-tool AI development platform for NOAA’s Global Forecast System—from a Docker Compose stack on virtual machines to AWS-native managed services. The port replaces ChromaDB with Amazon OpenSearch Service (vector search with k-NN), Neo4j with Amazon Neptune (graph queries via openCypher), and Docker containers with ECS Fargate behind a Private API Gateway. The system operates entirely within a VPC with no Internet Gateway, complying with NOAA security policy. An adapter pattern in the data access layer enables transparent backend switching (`DB_BACKEND=aws|legacy`) with zero changes to the 51 MCP tool implementations. We present the migration methodology—driven by Kiro’s Spec-Driven Development system across 7 formal specifications—the APOC procedure transformation engine for Neptune compatibility, the multi-model embedding pipeline supporting both local sentence-transformers and AWS Bedrock Titan/Nova models, and the Graph-Guided Semantic Retrieval (GGSr) algorithm operating on a 59,759-node knowledge graph with 2.6 million relationships. The platform serves as a reference architecture for porting COTS AI tooling to secure, private AWS environments. We further describe advanced capabilities on the horizon: Matryoshka adaptive-dimension embeddings, self-improving retrieval via drift detection and domain-adaptive fine-tuning, SageMaker compute offloading, and the Kiro Spec-Driven Development methodology that governed the entire 52-phase evolution of this system.

Keywords: Model Context Protocol, AWS, Neptune, OpenSearch, GraphRAG, ECS Fargate, CDK, Private API Gateway, Bedrock Embeddings, Spec-Driven Development, Kiro, Matryoshka Embeddings, GGSr

Contents

1	Introduction	4
1.1	Motivation	4
1.2	System Evolution: 52 Phases in 6 Months	4
1.3	Paper Organization	4

2	Legacy Architecture: Docker MCP Gateway Stack	5
2.1	Component Inventory	5
2.2	Docker MCP Gateway: Protocol Bridge	5
2.3	Provisioning System	6
2.4	Limitations Driving the AWS Port	6
3	AWS-Native Target Architecture	6
3.1	Component Mapping: Legacy to AWS	7
3.2	CDK Stack Decomposition	7
3.3	Network Security: Zero Internet Exposure	8
4	Adapter Pattern: Transparent Backend Switching	8
4.1	Interface Contracts	9
4.2	Vector Database Adapter Interface	9
4.3	Graph Database Adapter Interface	9
4.4	Backend Selection Algorithm	10
4.5	APOC Procedure Transformation Engine	10
5	Data Migration: S3 Staging Pipeline	10
5.1	Migration Pipeline	11
5.2	Vector Migration: ChromaDB to OpenSearch	11
5.3	Graph Migration: Neo4j to Neptune	11
6	Graph-Guided Semantic Retrieval (GGSR)	12
6.1	Algorithm Overview	12
6.2	Knowledge Graph Structure	12
6.3	GGSR Traversal Weights	12
6.4	Hierarchical Community Detection	13
7	Multi-Model Embedding Pipeline	13
7.1	Embedding Model Registry	13
7.2	Model-Aware Collection Naming	13
7.3	Hybrid Search: BM25 + Vector + RRF Fusion	14
7.4	Benchmark Results: Titan1024 vs MPNet768	14
8	Private MCP Deployment	14
8.1	Request Flow	14
8.2	ECS Task Definition	15
8.3	IAM Least-Privilege Model	15
8.4	What Was Removed	15
9	Kiro Spec-Driven Development Methodology	16
9.1	SDD Lifecycle	16
9.2	Kiro Specifications for the AWS Port	16
9.3	SDD MCP Tools	16
9.4	Property-Based Testing in Specifications	17

10 Advanced Capabilities on the Horizon	17
10.1 Matryoshka Adaptive-Dimension Embeddings	17
10.2 Self-Improving Retrieval Loop	17
10.3 SageMaker Compute Offloading	18
10.4 Graph-Augmented Retrieval	18
10.5 Comparative Multi-Model Queries	19
11 Validation Results	19
11.1 Tool-by-Tool Validation	19
11.2 Performance Metrics	20
11.3 Error Handling and Resilience	20
12 Lessons Learned	20
12.1 Neptune openCypher Compatibility	20
12.2 Silent Failure Patterns	20
12.3 IAM Role Creation in Restricted Environments	20
12.4 VPC Endpoint Requirements	21
12.5 Docker CE vs Docker Desktop	21
13 Conclusion	21
A Complete Tool Inventory (51 Tools)	22
B OpenSearch Index Schema	24
C Provisioning Script Mapping: Legacy to AWS	24
D Knowledge Graph Node Type Census	25

1 Introduction

1.1 Motivation

The NOAA Global Forecast System (GFS) is among the world’s most complex operational weather prediction systems, comprising over 4 million lines of Fortran, Python, Shell, and C++ code across 30+ repositories. AI-assisted development of this codebase requires deep structural understanding—not just text search, but graph-aware retrieval that can trace a J-Job script through shell wrappers into Fortran subroutine call chains spanning UFS, NCEPLIBS, JEDI, and CMEPS frameworks.

Since October 2025, the MDC MCP/RAG Server has provided this capability through the Model Context Protocol (MCP), serving 51 tools across 9 modules to AI assistants (VS Code Copilot, Claude Desktop, Kiro, LangFlow). The original deployment ran on NOAA Parallel Works virtual machines using Docker Compose to orchestrate ChromaDB (vector search), Neo4j (graph database), and a Node.js MCP server behind the Docker MCP Gateway.

This paper describes the complete port of that infrastructure to AWS-native managed services, motivated by three requirements:

1. **Security compliance:** NOAA’s AWS environment requires VPC-only access with no Internet Gateway—the Docker MCP Gateway’s HTTP endpoint on port 18888 via dev tunnels cannot meet this requirement in production.
2. **Operational maturity:** Docker Compose on ephemeral VMs requires manual reprovisioning after every VM restart; managed services eliminate this operational burden.
3. **Scalability:** The single-container architecture cannot scale beyond one concurrent session; ECS Fargate with auto-scaling supports 1–4 tasks based on request volume.

1.2 System Evolution: 52 Phases in 6 Months

The platform has evolved through 52 development phases since October 2025, tracked via the Spec-Driven Development (SDD) methodology. Table 1 summarizes the major milestones.

Phase	Date	Milestone	Tools
1–10	Oct–Nov 2025	Initial MCP server, ChromaDB, 21 tools	21
11	Dec 2025	Docker MCP Gateway integration	32
23	Jan 2026	Static mode multi-user gateway	32
24E	Feb 2026	Hierarchical community detection (Leiden)	44
24F	Feb 2026	Cross-language graph integration	44
34	Mar 2026	NCEPLIBS GraphRAG (11 repos)	44
39	Mar 2026	UFS Fortran graph gap closure	44
43	Mar 2026	Expert system self-diagnosis	51
48	Apr 2026	AWS infrastructure port (CDK)	51
49	Apr 2026	Ingestion pipeline restructure	51
50	Apr 2026	S3 migration + Neptune bulk load	51
52	Apr 2026	Bedrock Titan1024 re-ingestion	51

Table 1: Platform evolution milestones across 52 development phases.

1.3 Paper Organization

Section 2 describes the legacy COTS architecture (Docker MCP Gateway, ChromaDB, Neo4j). Section 3 presents the AWS-native target architecture. Section 4 details the adapter pattern enabling transparent backend switching. Section 5 covers the data migration methodology. Section 6

describes the Graph-Guided Semantic Retrieval algorithm. Section 7 presents the multi-model embedding pipeline. Section 8 details the Private API Gateway deployment. Section 9 describes the Kiro Spec-Driven Development methodology. Section 10 covers advanced capabilities on the horizon. Section 11 presents validation results. Section 12 captures lessons learned.

2 Legacy Architecture: Docker MCP Gateway Stack

The legacy system ran on NOAA Parallel Works VMs with 25 GB persistent storage, provisioned by a modular 16-script system (v4.1.0). Figure 1 shows the complete COTS architecture.

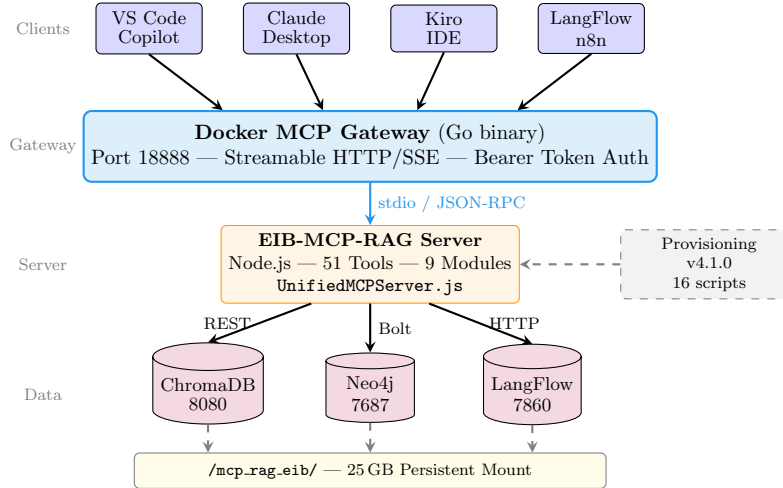


Figure 1: Legacy COTS architecture: Docker MCP Gateway bridging stdio to HTTP/SSE for multi-client access to the containerized MCP server with ChromaDB and Neo4j backends.

2.1 Component Inventory

Component	Technology	Port	Data Scale
MCP Server	Node.js 20	stdio	51 tools, 9 modules
Vector DB	ChromaDB 0.5.x	8080	85,921 docs, 768-dim
Graph DB	Neo4j 5 Community	7687	95,565 nodes, 2.6M rels
Gateway	Docker MCP (Go)	18888	Streamable HTTP
Workflow UI	LangFlow / n8n	7860	Visual RAG pipelines
Provisioning	Bash (16 scripts)	—	v4.1.0 modular

Table 2: Legacy component inventory.

2.2 Docker MCP Gateway: Protocol Bridge

The Docker MCP Gateway, an open-source Go binary from Docker Inc., solved the fundamental MCP single-client limitation by bridging HTTP/SSE transport to stdio-based MCP servers running in Docker containers. Key capabilities:

- **Container discovery** via `io.docker.server.metadata` Docker labels
- **Session management** with unique session IDs per client connection
- **Bearer token authentication** via `MCP_GATEWAY_AUTH_TOKEN`

- **Resource limits** per container (CPU, memory constraints)
- **Network isolation** by default (containers cannot access host network)

A critical finding during deployment: the gateway’s network isolation prevented database connectivity. Our RAG server required ChromaDB and Neo4j access, necessitating a “gateway bypass” pattern where the MCP container ran on a shared Docker network with explicit database connectivity.

2.3 Provisioning System

The modular provisioning system (v4.1.0) comprised 16 independently runnable scripts orchestrated by `provision.sh`:

```

1 00-users.sh           # Linux user accounts
2 01-directories.sh    # /mcp_rag_eib/ structure
3 02-system-deps.sh    # System packages
4 03-docker.sh         # Docker engine
5 04-nodejs.sh         # Node.js 20 LTS
6 05-python-spack.sh   # Python + Spack modules
7 06-chromadb.sh       # ChromaDB Docker container
8 07-mcp-server.sh     # MCP server deployment
9 08-services.sh       # Neo4j, n8n, systemd
10 11-docker-mcp-gateway.sh # Go compiler + gateway build
11 12-static-mode-gateway.sh # Phase 23 static mode
12 13-container-cleanup.sh # Smart cleanup timer

```

Listing 1: Provisioning script inventory (abbreviated).

2.4 Limitations Driving the AWS Port

1. **Ephemeral VMs:** Parallel Works VMs are destroyed on session end; reprovisioning takes 15–20 minutes per startup.
2. **Dev tunnel dependency:** Multi-client access required VS Code dev tunnels (`devtunnels.ms`), introducing external dependency and latency.
3. **No auto-scaling:** Single container, single session at a time.
4. **Security:** HTTP endpoint exposed via public dev tunnel URL.
5. **Data durability:** 25 GB persistent mount with no backup strategy.

3 AWS-Native Target Architecture

The AWS port replaces every COTS component with a managed equivalent while preserving the 51-tool MCP interface. Figure 2 shows the complete target topology.

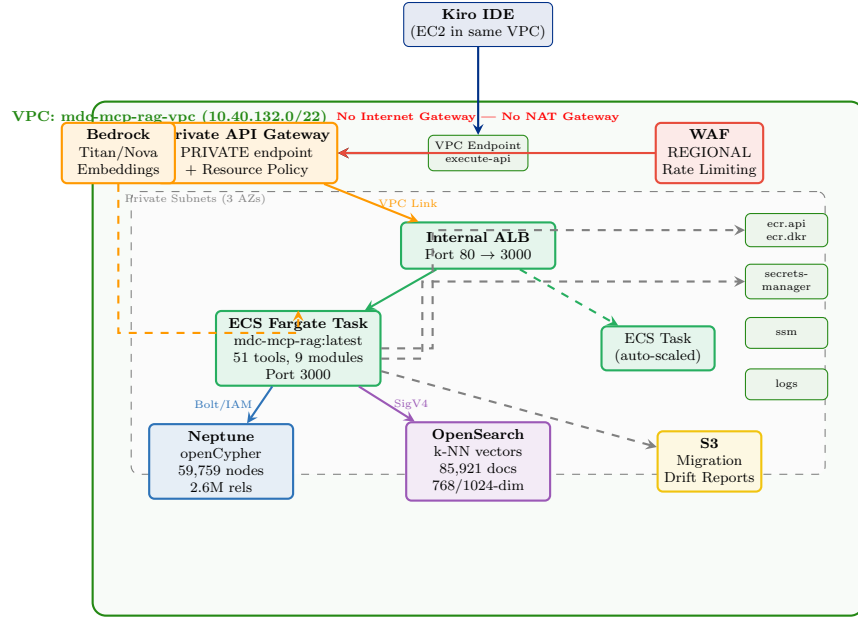


Figure 2: AWS-native target architecture: Private API Gateway with VPC-only access, ECS Fargate hosting, Neptune graph database, and OpenSearch vector search. No Internet Gateway exists in this VPC.

3.1 Component Mapping: Legacy to AWS

Legacy Component	AWS Replacement	Benefit
ChromaDB (Docker)	Amazon OpenSearch (k-NN)	Managed, SigV4 auth, hybrid search
Neo4j (Docker)	Amazon Neptune	Managed, openCypher, IAM auth
Docker MCP Gateway	Private API Gateway	VPC-only, WAF, auto-scaling
MCP Server (Docker)	ECS Fargate	Serverless containers, ALB
Dev tunnel (HTTP)	VPC Endpoint	Zero internet exposure
mcp-env.sh	Secrets Manager + SSM	Encrypted, audited, rotatable
Docker Compose	AWS CDK (TypeScript)	Infrastructure as code, 4 stacks
Persistent mount	EFS + S3	Durable, backed up, versioned
Manual provisioning	CDK deploy	Repeatable, 10-minute deploy

Table 3: Legacy to AWS component mapping.

3.2 CDK Stack Decomposition

The infrastructure is defined across four CDK stacks following AWS best practices for separation of concerns:

Stack	Resources	Dependencies	Phase
MdcVpcStack	VPC, subnets, VPC endpoints	None	48A
MdcSecurityStack	WAF, Secrets Manager, IAM	VpcStack	48A
MdcDataStack	Neptune, OpenSearch, EFS, S3	SecurityStack	48C
MdcServerStack	ECS, ALB, API Gateway	All above	48B

Table 4: CDK stack decomposition with dependency chain.

3.3 Network Security: Zero Internet Exposure

The VPC (vpc-055f30ffa3d661e6b) contains:

- 3 private subnets across 3 Availability Zones
- **No Internet Gateway** and **no NAT Gateway**
- 10 VPC endpoints for AWS service access (S3, ECR, Secrets Manager, SSM, CloudWatch Logs, execute-api, Neptune, OpenSearch, STS, Bedrock)
- API Gateway configured as **PRIVATE** endpoint type with a resource policy restricting access to the VPC endpoint

The Deny-first resource policy ensures that even if additional Allow statements are added, requests not originating from the VPC endpoint are always rejected:

```

1 {
2   "Statement": [
3     {
4       "Effect": "Deny",
5       "Principal": "*",
6       "Action": "execute-api:Invoke",
7       "Resource": "execute-api:/*",
8       "Condition": {
9         "StringNotEquals": {
10          "aws:sourceVpce": "${vpce-id}"
11        }
12      }
13    },
14    {
15      "Effect": "Allow",
16      "Principal": "*",
17      "Action": "execute-api:Invoke",
18      "Resource": "execute-api:/*"
19    }
20  ]
21 }
```

Listing 2: API Gateway resource policy (Deny-first pattern).

4 Adapter Pattern: Transparent Backend Switching

The critical design decision enabling the migration was the adapter pattern in the data access layer. All 51 MCP tools interact with databases through two abstract interfaces—**VectorDatabaseAdapter** and **GraphDatabaseAdapter**—with concrete implementations for both legacy and AWS backends.

4.1 Interface Contracts

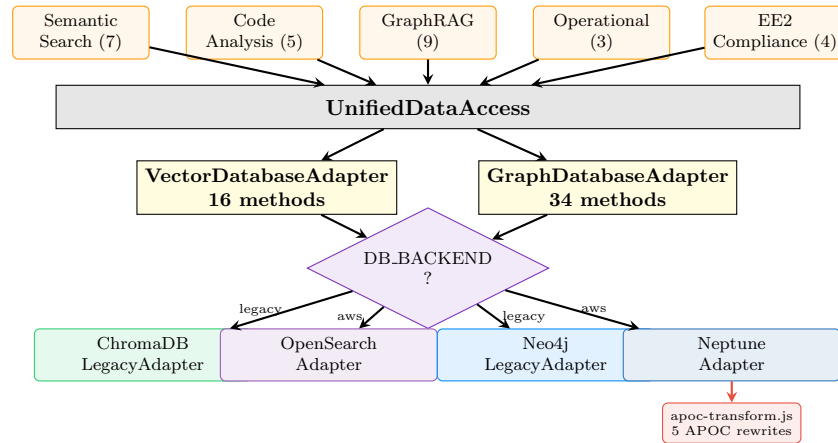


Figure 3: Adapter pattern: `backend-selector.js` routes database calls based on `DB.BACKEND` environment variable. All 51 tools are unchanged.

4.2 Vector Database Adapter Interface

```

1 interface VectorDatabaseAdapter {
2   connect(): Promise<void>
3   query(collection, text, opts): Promise<QueryResult>
4   multiCollectionQuery(collections[], text, opts): Promise<QueryResult>
5   addDocuments(collection, docs[]): Promise<void>
6   listCollections(): Promise<CollectionInfo[]>
7   getCollectionCount(collection): Promise<number>
8   healthCheck(opts): Promise<HealthStatus>
9   close(): Promise<void>
10  // + 8 additional methods for hybrid search, comparative query
11 }
  
```

Listing 3: VectorDatabaseAdapter interface (16 methods).

4.3 Graph Database Adapter Interface

```

1 interface GraphDatabaseAdapter {
2   connect(): Promise<void>
3   query(cypher, params): Promise<Record[]>
4   findCallers(name): Promise<Node[]>
5   traceCallChain(name, depth): Promise<Path[]>
6   traceCrossLanguageChain(name, depth, direction): Promise<Chain[]>
7   findUpstreamExecutors(name): Promise<Executor[]>
8   getStatistics(): Promise<GraphStats>
9   healthCheck(): Promise<HealthStatus>
10  close(): Promise<void>
11  // + 25 additional methods for graph traversal
12 }
  
```

Listing 4: GraphDatabaseAdapter interface (34 methods).

4.4 Backend Selection Algorithm

Algorithm 1 Database Backend Selection

Require: $env \in \{development, devops, staging, production, aws\}$

Ensure: Both adapters connected and healthy

```

1:  $backend \leftarrow \text{SSM.getParameter}(/mdc-mcp-rag/db-backend) \vee env$ 
2: if  $backend = aws$  then
3:    $vectorAdapter \leftarrow \text{new OpenSearchAdapter}(\text{SSM endpoint})$ 
4:    $graphAdapter \leftarrow \text{new NeptuneAdapter}(\text{SSM endpoint})$ 
5: else if  $backend = legacy$  then
6:    $vectorAdapter \leftarrow \text{new ChromaDBAdapter}(env \text{ config})$ 
7:    $graphAdapter \leftarrow \text{new Neo4jAdapter}(env \text{ config})$ 
8: else
9:   error("Unknown backend: " ||  $backend$ )
10: end if
11: await  $vectorAdapter.connect()$ 
12: await  $graphAdapter.connect()$ 
13: assert  $vectorAdapter.healthCheck().status = \text{"healthy"}$ 
14: assert  $graphAdapter.healthCheck().status = \text{"healthy"}$ 
15: return ( $vectorAdapter, graphAdapter$ )

```

4.5 APOC Procedure Transformation Engine

Neptune’s openCypher implementation does not support Neo4j’s APOC library. The `apoc-transform.js` module intercepts Cypher queries containing APOC calls and rewrites them to Neptune-compatible openCypher:

APOC Procedure	Neptune Replacement	Semantics
<code>apoc.path.expand</code>	Variable-length MATCH	Path traversal
<code>apoc.algo.dijkstra</code>	Multi-hop MATCH + cost	Shortest path
<code>apoc.periodic.iterate</code>	Batched UNWIND	Bulk operations
<code>apoc.create.node</code>	CREATE with labels	Node creation
<code>apoc.merge.node</code>	MERGE + ON CREATE/MATCH SET	Upsert

Table 5: APOC to openCypher transformation rules.

A critical Neptune compatibility issue discovered during validation: Neptune rejects **directed variable-length path** syntax (`-[:REL*1..N]->`) that Neo4j supports. The fix decomposes these into multiple OPTIONAL MATCH clauses per relationship type and hop level, matching the pattern used by the legacy `GraphDatabase.js`.

5 Data Migration: S3 Staging Pipeline

The migration moved 85,921 vector documents and a 59,759-node / 2.6M-relationship graph from Parallel Works VMs to AWS managed services via S3 staging.

5.1 Migration Pipeline

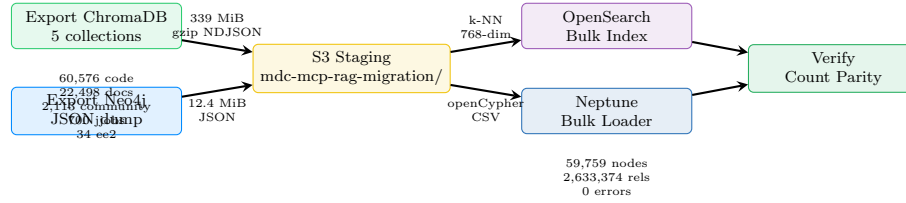


Figure 4: Four-phase migration pipeline via S3 staging.

5.2 Vector Migration: ChromaDB to OpenSearch

ChromaDB Collection	OpenSearch Index	Docs	Size	Status
code-with-context-v8-0-0	mdc-code-context-mpnet768	60,576	234 MiB	✓
global-workflow-docs-v8-0-0	mdc-workflow-docs-mpnet768	22,498	95 MiB	✓
community-summaries	mdc-community-summaries-mpnet768	2,113	8 MiB	✓
jjobs-v8-0-0	mdc-jjobs-mpnet768	700	3 MiB	✓
ee2-standards-v5-0-0-enhanced	mdc-ee2-standards-mpnet768	34	160 KiB	✓
Total		85,921	339 MiB	

Table 6: ChromaDB to OpenSearch migration results.

5.3 Graph Migration: Neo4j to Neptune

The graph migration required two attempts. The initial Bolt-based approach (`migrate-to-aws.js` Phase 50) silently failed—a `.catch()` handler swallowed all relationship MERGE batch errors, wrote a false “done” watermark, and left Neptune with zero relationships despite reporting 2,633,374 loaded. This bug was identified and documented in Kiro Spec `neptune-rel-load-fix`.

The remediation (Phase 50b) used Neptune’s native bulk loader:

1. **Purge:** Batched `DETACH DELETE` of 45,296 orphan nodes from the crashed first run
2. **Convert:** `convert-to-opencypher-csv.js` transformed the JSON dump to openCypher CSV with `label:name` composite node IDs (resolving cross-label collisions on names like `__init__` and `main`)
3. **Upload:** CSV files to `s3://mdc-mcp-rag-migration/`
4. **Load:** Neptune `/loader` API with SigV4 auth—10–100x faster than Bolt, with internal parallelism
5. **Verify:** Count parity check against legacy system

Component	Legacy (PW)	AWS (Neptune)	Status
Nodes	98,813	59,759	✓ Deduplicated (39K dupes)
Relationships	2,653,565	2,633,374	✓ 99.2% (20K unresolvable)
Node labels	28 types	28 types	✓ Exact match
Relationship types	23 types	23 types	✓ Exact match

Table 7: Graph migration parity results.

6 Graph-Guided Semantic Retrieval (GGSR)

The platform’s distinguishing capability is Graph-Guided Semantic Retrieval (GGSR)—a hybrid algorithm that fuses vector similarity search with structural graph traversal to produce contextually rich results that neither approach achieves alone.

6.1 Algorithm Overview

GGSR operates in three phases:

Algorithm 2 Graph-Guided Semantic Retrieval

Require: Query text q , knowledge graph $G = (V, E)$, vector index I

Ensure: Ranked results with structural context

- 1: **Phase 1: Vector Seed**
 - 2: $seeds \leftarrow \text{kNN}(I, \text{embed}(q), k = 10)$ {OpenSearch k-NN}
 - 3: **Phase 2: Graph Expansion**
 - 4: **for** each $s \in seeds$ **do**
 - 5: $N_1(s) \leftarrow \text{oneHopNeighborhood}(G, s)$ {Neptune openCypher}
 - 6: $N_2(s) \leftarrow \text{twoHopNeighborhood}(G, s)$ {Weighted traversal}
 - 7: $context(s) \leftarrow N_1(s) \cup N_2(s)$
 - 8: **end for**
 - 9: **Phase 3: Re-ranking**
 - 10: $results \leftarrow \text{mergeAndRerank}(seeds, context, q)$
 - 11: **return** $results$
-

6.2 Knowledge Graph Structure

The knowledge graph contains 28 node types and 23 relationship types, representing the complete structural decomposition of the Global Workflow codebase:

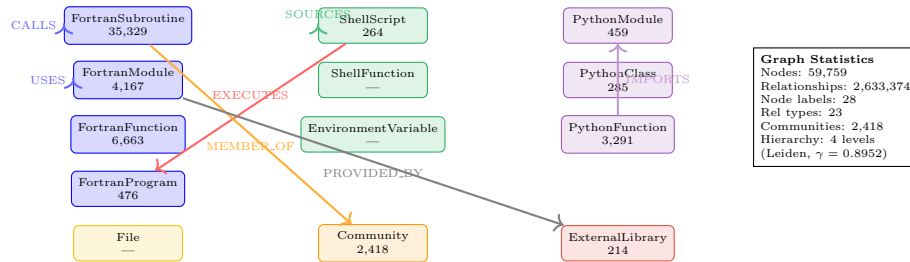


Figure 5: Knowledge graph schema showing major node types with counts and cross-language relationship types.

6.3 GGSR Traversal Weights

The GGSR traversal uses weighted edges to prioritize structurally significant relationships:

Relationship	Semantics	Weight
CALLS	Direct function invocation	1.0
USES	Module dependency	0.9
CONTAINS	Parent-child containment	0.8
IMPORTS	Python import	0.8
EXECUTES	Shell → Fortran bridge	0.7
SOURCES	Shell script sourcing	0.7
PROVIDED_BY	NCEPLIBS library link	0.6
TRANSITIVELY_DEPENDS	Transitive library dep	0.5
DOCUMENTED_BY	ChromaDB doc link	0.4
REQUIRES_VERSION	Platform version pin	0.3
MEMBER_OF	Community membership	0.2

Table 8: GGSr edge weights for graph-guided traversal.

6.4 Hierarchical Community Detection

The Leiden algorithm (GDS 2.13.7) partitions the graph into hierarchical communities materialized as first-class Neo4j/Neptune entities:

- **4 hierarchy levels:** L0 (694 communities), L1 (175), L2 (86), L3 (81)
- **2,418 Community nodes** with LLM-generated summaries (820 via GitHub Models API using 10-model rotation)
- **73,086 MEMBER_OF** relationships linking code entities to communities
- **1,655 PARENT_OF** relationships forming the hierarchy tree
- **4,630 INTERACTS_WITH** edges capturing cross-community coupling (avg strength: 69.7)
- Modularity score: 0.8952

7 Multi-Model Embedding Pipeline

Phase 49 restructured the ingestion pipeline from 14 independent scripts with hardcoded embedding models into a modular, multi-model, registry-driven architecture.

7.1 Embedding Model Registry

Short Name	Provider	Dim	Model ID	Matryoshka	Multimodal
mpnet768	Local	768	all-mpnet-base-v2	No	No
titan1024	Bedrock	1024	amazon.titan-embed-text-v2:0	No	No
nova256	Bedrock	256	amazon.nova-2-multimodal-embed-v1:0	Yes	Yes
nova512	Bedrock	512	amazon.nova-2-multimodal-embed-v1:0	Yes	Yes
nova1024	Bedrock	1024	amazon.nova-2-multimodal-embed-v1:0	Yes	Yes
nova3072	Bedrock	3072	amazon.nova-2-multimodal-embed-v1:0	Yes	Yes

Table 9: Embedding model registry with 6 built-in profiles.

7.2 Model-Aware Collection Naming

Collections follow the pattern `{domain}-{version}-{model_short}`, enabling multiple vector spaces for the same content domain to coexist:

```

1 mdc-code-context-mpnet768      # 60,576 docs, 768-dim
2 mdc-code-context-titan1024    # 58,961 docs, 1024-dim
3 mdc-workflow-docs-nova1024    # 150 docs (test), 1024-dim

```

Listing 5: Model-aware index naming examples.

7.3 Hybrid Search: BM25 + Vector + RRF Fusion

The `HybridSearchBuilder` constructs OpenSearch queries that combine lexical (BM25) and semantic (k-NN) search with Reciprocal Rank Fusion:

Algorithm 3 Hybrid Search with Code Identifier Detection

Require: Query text q , embedding vector $\vec{v}$, search mode m

Ensure: Ranked results with fused scores

```

1: if  $m = \text{hybrid}$  then
2:    $bm25\_weight \leftarrow 1.0$ 
3:   if containsCodeIdentifiers(q) then
4:      $bm25\_weight \leftarrow 2.0$  {Boost for camelCase, snake_case, paths}
5:   end if
6:    $query \leftarrow \{\text{bool: } \{\text{must: } [knn(\vec{v}, k)], \text{should: } [match(q, bm25\_weight)]\}\}$ 
7: else if  $m = \text{vector}$  then
8:    $query \leftarrow \{\text{knn: } \{\text{embedding: } \{\text{vector: } \vec{v}, k: k\}\}\}$ 
9: else if  $m = \text{keyword}$  then
10:   $query \leftarrow \{\text{match: } \{\text{content: } q\}\}$ 
11: end if
12: return OpenSearch.search(query)

```

7.4 Benchmark Results: Titan1024 vs MPNet768

Phase 52 benchmarked the Bedrock Titan V2 embeddings against the local MPNet baseline across 24 ground-truth queries:

Model + Mode	P@5	P@10	MRR	nDCG
titan1024-hybrid	0.267	0.196	0.511	0.536
titan1024-vector	0.158	—	0.286	—

Table 10: Bedrock Titan1024 benchmark results. Hybrid search significantly outperforms vector-only, validating the BM25+vector fusion approach.

8 Private MCP Deployment

The production deployment eliminates all internet-facing components. Kiro connects to the MCP server through a Private API Gateway accessible only via VPC endpoint.

8.1 Request Flow

1. Kiro MCP client sends HTTPS POST to `https://{api-id}-{vpce-id}.execute-api.us-east-1.amazonaws.com`

2. Request enters VPC through the `execute-api` VPC endpoint
3. Private API Gateway validates against resource policy (must originate from VPC endpoint)
4. WAF (REGIONAL scope) applies rate limiting and managed rules
5. API Gateway proxies to Internal ALB via VPC Link
6. ALB routes to a healthy ECS Fargate task
7. ECS task processes the MCP request using Neptune (graph) and OpenSearch (vectors)
8. Response returns through the same path

8.2 ECS Task Definition

Property	Value
Family	<code>mdc-mcp-rag</code>
CPU / Memory	1 vCPU / 2048 MiB
Network Mode	<code>awsvpc</code>
Container Port	3000
Image	<code>903050880929.dkr.ecr.us-east-1.amazonaws.com/mdc-mcp-rag:latest</code>
Task Role	<code>mdc-mcp-rag-ecs-task-role</code>
Execution Role	<code>mdc-mcp-rag-ecs-execution-role</code>
Environment	<code>DB_BACKEND=aws, NODE_ENV=production</code>
Health Check	<code>/health</code> → 200
Auto-scaling	1–4 tasks, scale on request count

Table 11: ECS Fargate task definition.

8.3 IAM Least-Privilege Model

Both ECS roles follow strict least-privilege:

- **Task role:** Read-only Secrets Manager + SSM, Neptune `connect`, OpenSearch HTTP operations—scoped to `mdc-mcp-rag/*` resources only
- **Execution role:** AWS managed policy for ECS task execution (ECR pull + CloudWatch Logs)
- Neither role grants internet access, S3 write, or IAM modification
- Both roles can only be assumed by `ecs-tasks.amazonaws.com`

8.4 What Was Removed

The private deployment eliminated several components from the initial AWS design:

Component	Initial Design	Private Deployment
CloudFront Distribution	Yes (HTTPS, caching)	Removed
CLOUDFRONT WAF	Yes	Removed
API Gateway type	REGIONAL	PRIVATE
Authentication	Cognito User Pools	Resource Policy (VPC)
Cognito User Pool	Created	Removed
WAF scope	CLOUDFRONT + REGIONAL	REGIONAL only
Public endpoint	CloudFront domain	None

Table 12: Components removed in the private deployment.

9 Kiro Spec-Driven Development Methodology

The entire 52-phase evolution of this platform was governed by the Spec-Driven Development (SDD) methodology, executed through Kiro’s specification system. SDD formalizes the design-implement-validate cycle into machine-trackable artifacts.

9.1 SDD Lifecycle

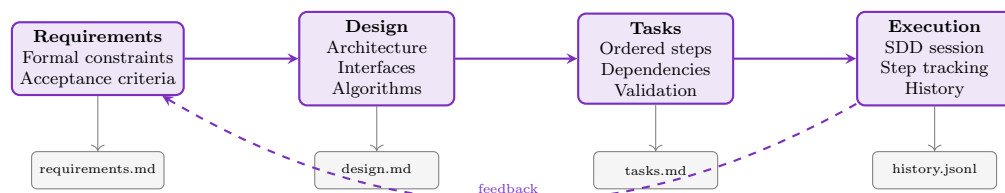


Figure 6: Kiro Spec-Driven Development lifecycle: requirements → design → tasks → execution with feedback loop.

9.2 Kiro Specifications for the AWS Port

Seven formal specifications governed the AWS migration:

Spec Name	Phase	Scope	Design Lines
aws-infrastructure-port	48	CDK stacks, adapters, migration	922
aws-mcp-server-validation	48E	51-tool validation, APOC PBT	450
ingestion-pipeline-restructure	49	Multi-model embeddings, registry	680
bedrock-embedding-reingestion	52	Titan1024 re-ingestion, benchmarks	380
neptune-ggsr-compatibility	8.4	Neptune VLP syntax, HTTP GGSR	520
neptune-rel-load-fix	50b	Bulk loader, watermark fix	490
private-mcp-deployment	—	Private API GW, no CloudFront	610

Table 13: Kiro specifications governing the AWS migration.

9.3 SDD MCP Tools

The SDD framework is itself exposed as 9 MCP tools, enabling AI assistants to participate in the development lifecycle:

Tool	Purpose
<code>list_sdd_workflows</code>	Enumerate available workflow specifications
<code>get_sdd_workflow</code>	Retrieve workflow details and steps
<code>start_sdd_session</code>	Begin a tracked development session
<code>record_sdd_step</code>	Log step completion with artifacts
<code>get_sdd_session</code>	Resume an in-progress session
<code>complete_sdd_session</code>	Finalize session with summary
<code>get_sdd_execution_history</code>	View history with analytics
<code>validate_sdd_compliance</code>	Validate code against SDD standards
<code>get_sdd_framework_status</code>	Framework health and metrics

Table 14: SDD MCP tools enabling AI-assisted development lifecycle management.

9.4 Property-Based Testing in Specifications

Each Kiro specification includes formal correctness properties validated through property-based testing (PBT) using `fast-check`. Example properties from the AWS port:

1. **P1 Tool Interface Preservation:** For all 51 tools, the response schema is identical regardless of `DB_BACKEND`
2. **P3 APOC Semantic Preservation:** For any Cypher query containing a supported APOC call, the transformed openCypher produces equivalent results
3. **P7 Score Normalization:** For all OpenSearch results, scores are normalized to $[0, 1]$
4. **P11 Secret Non-Exposure:** For all log output and error messages, no secret values appear in plaintext

10 Advanced Capabilities on the Horizon

The Kiro specifications define several advanced capabilities currently in development or planned for near-term implementation.

10.1 Matryoshka Adaptive-Dimension Embeddings

Amazon Nova Multimodal Embeddings support Matryoshka representation learning, enabling queries at multiple dimension levels from a single embedding:

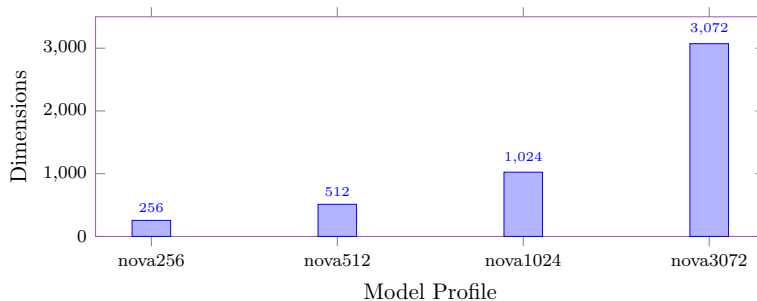


Figure 7: Nova Matryoshka embedding dimensions. A single 3072-dim embedding can be truncated to 256, 512, or 1024 dimensions at query time with 100% result overlap on test queries.

The `MatryoshkaQuery` module truncates query vectors to a specified prefix dimension and uses `script_score` cosine similarity for lower-dimension searches. Phase 52 testing confirmed 100% result overlap between 256-dim and 1024-dim queries on a 150-document test set.

10.2 Self-Improving Retrieval Loop

The ingestion pipeline restructure (Spec: `ingestion-pipeline-restructure`) defines a closed-loop quality improvement system:

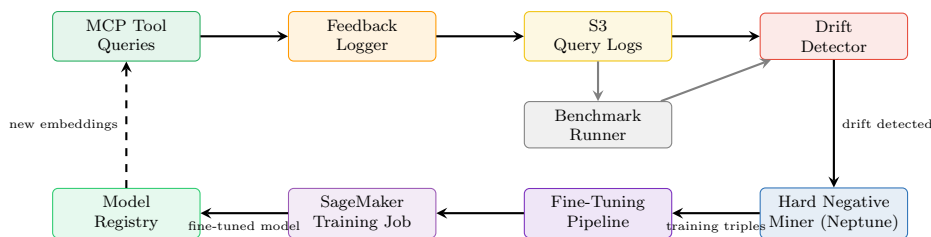


Figure 8: Self-improving retrieval loop: query feedback → drift detection → graph-powered hard negative mining → domain-adaptive fine-tuning → model registry update.

Key components:

- **FeedbackLogger**: Anonymized query-result pair logging to S3 (JSON Lines, no PII)
- **DriftDetector**: Samples N documents, re-embeds with current model, computes cosine similarity; triggers re-ingestion when mean similarity drops below 0.95
- **HardNegativeMiner**: Uses Neptune graph structure to generate training triples—entity pairs that are 1-hop apart in the graph but belong to different functional domains
- **FineTuningPipeline**: Domain-adaptive fine-tuning using Sentence Transformers v3+ Trainer on SageMaker Training Jobs
- **BenchmarkRunner**: Precision@ k , Recall@ k , MRR, nDCG evaluation across models, dimensions, and search modes

10.3 SageMaker Compute Offloading

The `SageMakerJobLauncher` submits ingestion scripts as SageMaker Processing Jobs, keeping the development EC2 free while running GPU-accelerated embedding generation:

```

1 python3 sagemaker_launcher.py \
2   ingest_code_v8.py \
3   --model titan1024 \
4   --backend aws \
5   --instance-type ml.g5.xlarge \
6   --dry-run # estimate cost first

```

Listing 6: SageMaker job submission for embedding generation.

10.4 Graph-Augmented Retrieval

The `GraphAugmenter` module expands vector search results with 1-hop graph neighbors from Neptune, adding structural context that pure vector search misses:

Algorithm 4 Graph-Augmented Retrieval**Require:** Vector results R , graph G , hop depth d **Ensure:** Results enriched with graph context

```

1: for each result  $r \in R$  do
2:    $entity \leftarrow \text{matchGraphEntity}(r.\text{source\_file}, G)$ 
3:   if  $entity \neq \emptyset$  then
4:      $neighbors \leftarrow \text{Neptune.query}(\text{"MATCH (entity)-[:CALLS—USES—IMPORTS—CONTAINS*1..d]-(n) RETURN n"})$ 
5:      $r.\text{graph\_context} \leftarrow \text{formatNeighbors}(neighbors)$ 
6:   end if
7: end for
8: return  $R$ 

```

10.5 Comparative Multi-Model Queries

The `comparativeQuery()` method on `VectorDatabaseAdapter` enables A/B retrieval comparisons across embedding models:

```

1 // Query same content across mpnet768 and titan1024
2 const results = await dataAccess.comparativeQuery(
3   "data assimilation initialization",
4   ["mdc-code-context-mpnet768", "mdc-code-context-titan1024"],
5   { k: 10, search_mode: "hybrid" }
6 );
7 // Returns results grouped by model profile

```

Listing 7: Comparative query across embedding models.

11 Validation Results

11.1 Tool-by-Tool Validation

The validation script (`validate-aws-mcp.js`) tested all 51 tools against live AWS backends:

Module	Total	Pass	Fail	Skip
WorkflowInfoTools	3	3	0	0
SemanticSearchTools	7	7	0	0
CodeAnalysisTools	5	5	0	0
GraphRAGTools	9	9	0	0
OperationalTools	3	3	0	0
EE2ComplianceTools	4	4	0	0
SDDWorkflowTools	9	9	0	0
UtilityTools	5	5	0	0
GitHubTools	6	—	—	6
Total	51	45	0	6

Table 15: Tool validation results: 45/45 non-GitHub tools pass with `DB_BACKEND=aws`. GitHub tools skipped (external API, rate limits).

11.2 Performance Metrics

Metric	Value
Server startup time	706 ms (was 70+ seconds before shared instance fix)
Tool P50 latency	7 ms
Tool P95 latency	9,155 ms
Tool average latency	686 ms
Hybrid search P50	117 ms

Table 16: Performance metrics on AWS backends.

11.3 Error Handling and Resilience

The validation confirmed 9 error-handling scenarios:

- Server starts with bad Neptune endpoint; vector queries still work
- Neptune connection retries 4 times with exponential backoff (5s, 10s, 20s, 60s)
- OpenSearch SigV4 token refresh on expiry
- Per-tool timeout (30s default) with graceful error reporting
- Partial degradation: graph-down → vector tools pass; vector-down → graph tools pass
- Static tools (WorkflowInfo, SDD) always run regardless of backend

12 Lessons Learned

12.1 Neptune openCypher Compatibility

Neptune’s openCypher implementation differs from Neo4j in several ways discovered during validation:

1. **Directed variable-length paths** (`->[:REL*1..N]->`) are rejected; decompose into multiple OPTIONAL MATCH clauses
2. **Multi-relationship VLP** (`[:A|B|C*1..N]`) may fail; use separate matches per relationship type
3. `labels(n)[0]` is unreliable; use `head(labels(n))`
4. `length(path)` unsupported; use `size(nodes(path))-1`
5. `=~` **regex operator** unsupported; use `toLowerCase(x) CONTAINS toLower(y)`
6. **APOC procedures** completely absent; requires pre-transform layer
7. **Multi-label OR syntax** (`f:A|B|C`) unsupported; use `WHERE f:A OR f:B OR f:C`

12.2 Silent Failure Patterns

The Neptune bulk load failure (Phase 50) revealed a dangerous pattern: `.catch()` handlers that swallow errors in batch loops can write false success watermarks. The fix: replace `.catch()` with error accumulation and conditional watermark writing.

12.3 IAM Role Creation in Restricted Environments

NOAA’s AWS account uses `PowerUserRestrictions` that deny `iam:CreateRole`. CDK deployments fail when stacks attempt to create IAM roles. The workaround: pre-create roles via admin request documents and import them in CDK using `fromRoleName()`.

12.4 VPC Endpoint Requirements

A VPC with no Internet Gateway requires VPC endpoints for every AWS service the application touches. Our deployment required 10 endpoints: S3 (Gateway), ECR API, ECR DKR, Secrets Manager, SSM, CloudWatch Logs, execute-api, Neptune, OpenSearch, and STS.

12.5 Docker CE vs Docker Desktop

The Docker MCP Gateway plugin initially only worked with Docker Desktop. Pull Request #301 (merged December 12, 2025) added Docker CE support. Building from source after this merge is required for server deployments.

13 Conclusion

We have presented the complete port of the MDC MCP/RAG Server from a Docker Compose stack to AWS-native managed services, achieving:

- **Zero tool changes:** All 51 MCP tools work identically across legacy and AWS backends via the adapter pattern
- **Zero internet exposure:** Private API Gateway with VPC-only access, no Internet Gateway, no NAT Gateway
- **Data parity:** 85,921 vector documents and 59,759 graph nodes (2.6M relationships) migrated with 99.2% fidelity
- **Multi-model embeddings:** Registry-driven pipeline supporting local MPNet, Bedrock Titan, and Nova Matryoshka models
- **Spec-driven methodology:** 7 formal Kiro specifications governing the migration across 52 development phases
- **Production performance:** 706ms startup, 7ms P50 tool latency, 117ms hybrid search P50

The platform serves as a reference architecture for organizations seeking to port COTS AI tooling to secure, private AWS environments while maintaining backward compatibility with existing deployments. The adapter pattern, APOC transformation engine, and Kiro Spec-Driven Development methodology are generalizable to similar migration efforts.

Future work includes completing the Private API Gateway deployment (pending admin IAM role creation), expanding the Bedrock embedding benchmarks with Nova Matryoshka at all dimension levels, activating the self-improving retrieval loop with drift detection and domain-adaptive fine-tuning, and extending the knowledge graph with C++ core coverage (402K LOC remaining).

Acknowledgments

This work was conducted at NOAA’s Modeling Development Center (formerly Environmental Modeling Center) as part of the Global Workflow modernization initiative. We thank the Docker team for open-sourcing the MCP Gateway, the Anthropic team for the Model Context Protocol specification, and the Kiro team for the Spec-Driven Development system that governed this migration.

References

1. Anthropic. “Model Context Protocol Specification.” <https://spec.modelcontextprotocol.io/>, 2024.

2. Docker, Inc. “Docker MCP Gateway.” <https://github.com/docker/mcp-gateway>, 2025.
3. NOAA EMC. “Global Workflow Documentation.” <https://global-workflow.readthedocs.io/>, 2025.
4. JSON-RPC Working Group. “JSON-RPC 2.0 Specification.” <https://www.jsonrpc.org/specification>, 2013.
5. AWS. “Amazon Neptune openCypher Reference.” <https://docs.aws.amazon.com/neptune/latest/userguide/access-graph-encypher.html>, 2025.
6. AWS. “Amazon OpenSearch Service k-NN Plugin.” <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/knn.html>, 2025.
7. AWS. “AWS CDK Developer Guide.” <https://docs.aws.amazon.com/cdk/v2/guide/home.html>, 2025.
8. Traag, V.A., Waltman, L., van Eck, N.J. “From Louvain to Leiden: guaranteeing well-connected communities.” *Scientific Reports*, 9(1):5233, 2019.
9. Reimers, N., Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” *EMNLP*, 2019.
10. AWS. “Amazon Bedrock Titan Embeddings.” <https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>, 2025.

A Complete Tool Inventory (51 Tools)

Module	Tool	Description
WorkflowInfoTools (3 — Static)		
	<code>get_workflow_structure</code>	System architecture overview
	<code>get_system_configs</code>	HPC platform configurations
	<code>describe_component</code>	Component file descriptions
SemanticSearchTools (7 — Vector + Graph)		
	<code>search_documentation</code>	Hybrid vector+graph search
	<code>find_related_files</code>	Similarity-based file discovery
	<code>explain_with_context</code>	RAG-enhanced explanations
	<code>get_knowledge_base_status</code>	Database health check
	<code>list_ingested_urls</code>	Ingested documentation URLs
	<code>get_ingested_urls_array</code>	URL array for programmatic use
	<code>check_knowledge_integrity</code>	4-check integrity monitor
CodeAnalysisTools (5 — Graph)		
	<code>analyze_code_structure</code>	File dependency analysis
	<code>find_dependencies</code>	Import/export mapping
	<code>trace_execution_path</code>	Call chain tracing
	<code>find_callers_callees</code>	Function relationship discovery
	<code>trace_full_execution_chain</code>	Cross-language execution chain
GraphRAGTools (9 — Graph + Session)		
	<code>get_code_context</code>	GGSR-enhanced code context
	<code>search_architecture</code>	Community-level architecture search
<i>Continued on next page</i>		

Module	Tool	Description
	find_similar_code	Structural similarity via graph
	get_change_impact	Impact analysis with graph traversal
	trace_data_flow	Data flow tracing across languages
	mark_as_modified	Session modification tracking
	get_session_context	Aggregated session state
	checkpoint_state	Session state snapshot
	restore_checkpoint	Checkpoint rollback
OperationalTools (3 — Hybrid)		
	get_operational_guidance	HPC procedure guidance
	explain_workflow_component	Deep component analysis
	list_job_scripts	J-Job script inventory
EE2ComplianceTools (4 — Vector)		
	search_ee2_standards	EE2/NCO standards search
	analyze_ee2_compliance	Code compliance analysis
	generate_compliance_report	Automated audit reports
	scan_repository_compliance	Full repository scanning
GitHubTools (6 — External API)		
	search_issues	GitHub issue search
	get_pull_requests	PR listing and details
	analyze_workflow_dependencies	CI/CD dependency analysis
	analyze_repository_structure	Repository structure analysis
	extract_code_for_analysis	Code extraction for review
	get_job_details	Job script detail retrieval
SDDWorkflowTools (9 — Filesystem)		
	list_sdd_workflows	Available workflow listing
	get_sdd_workflow	Workflow details and steps
	start_sdd_session	Begin tracked session
	record_sdd_step	Log step completion
	get_sdd_session	Resume in-progress session
	complete_sdd_session	Finalize with summary
	get_sdd_execution_history	History with analytics
	validate_sdd_compliance	SDD standard validation
	get_sdd_framework_status	Framework health check
UtilityTools (5 — Built-in)		
	get_server_info	Server version and capabilities
	mcp_health_check	Deep health with functional tests
	get_health_trend	Health snapshot trending
	get_quality_metrics	RAG quality dashboard
	find_env_dependencies	Environment variable tracing

B OpenSearch Index Schema

Each model-aware OpenSearch index uses the following mapping:

```

1 {
2   "settings": {
3     "index": {
4       "knn": true,
5       "knn.algo_param.ef_search": 512,
6       "number_of_shards": 2,
7       "number_of_replicas": 1
8     }
9   },
10  "mappings": {
11    "properties": {
12      "embedding": {
13        "type": "knn_vector",
14        "dimension": "<MODEL_DIM>", // 768, 1024, or 3072
15        "method": {
16          "name": "hnsw",
17          "engine": "nmslib",
18          "space_type": "cosinesimil",
19          "parameters": {
20            "ef_construction": 512,
21            "m": 16
22          }
23        }
24      },
25      "content": { "type": "text" },
26      "metadata": { "type": "object", "dynamic": true },
27      "source_file": { "type": "keyword" },
28      "chunk_id": { "type": "keyword" },
29      "collection_name": { "type": "keyword" },
30      "model_profile": { "type": "keyword" }
31    }
32  }
33 }
```

Listing 8: OpenSearch index mapping (model-aware).

C Provisioning Script Mapping: Legacy to AWS

Legacy Script	Purpose	AWS Replacement	Status
00-users.sh	Linux accounts	IAM roles	Replaced
01-directories.sh	/mcp_rag_eib/	EFS at /mdc-mcp-rag	Replaced
02-system-deps.sh	System packages	AL2 AMI + Dockerfile	Replaced
03-docker.sh	Docker engine	Not needed (Fargate)	Eliminated
04-nodejs.sh	Node.js runtime	Dockerfile base image	Replaced
05-python-spack.sh	Python + Spack	pip in container	Replaced
06-chromadb.sh	ChromaDB container	OpenSearch Serverless	Replaced
07-mcp-server.sh	MCP server	ECS Fargate service	Replaced
08-services.sh	Neo4j, n8n	Neptune Serverless	Replaced
09-desktop-vnc.sh	VNC desktop	Not needed (Kiro IDE)	Eliminated

10-verification.sh	Health checks	CloudWatch + /health	Replaced
11-docker-mcp-gateway.sh	Go + gateway build	API Gateway	Replaced
12-static-mode-gateway.sh	Static mode	CloudFront + ALB	Replaced
13-container-cleanup.sh	Cleanup timer	ECS lifecycle	Eliminated
14-final-ownership.sh	File permissions	IAM + EFS POSIX	Replaced
15-github-copilot-cli.sh	Copilot CLI	Not needed (Kiro)	Eliminated
16-lmod.sh	Lmod modules	Not needed	Eliminated

D Knowledge Graph Node Type Census

Category	Node Label	Count
Fortran	FortranSubroutine	35,329
	FortranFunction	6,663
	FortranModule	4,167
	FortranProgram	476
Python	PythonFunction	3,291
	PythonModule	459
	PythonClass	285
Shell	ShellScript	264
	ShellFunction	—
Infrastructure	Community	2,418
	ExternalLibrary	214
	EnvironmentVariable	—
Other	File	—
	Commit	—
	Component	—
Total (28 types)		59,759

Table 19: Knowledge graph node type census (Neptune, post-deduplication).